La structure de `vpp_acceleration`, simplement

Chercher la section du C-foil qui fait aller le bateau le plus vite

Note pédagogique · état du code du 8e run (29/10/2026)

Ce que cette note explique. Toute la chaîne, des codes d'hydrodynamique jusqu'à la meilleure section, en quatre briques : la section (§ 2), le modèle des efforts (§ 3), la VPP (§ 4) et la boucle qui les relie (§ 5). Chaque brique dit *ce qu'elle fait*, *pourquoi*, et *où elle est dans le code*. Les détails du choix des nouveaux calculs sont dans `selection_points_vpp.tex` ; la description complète, dans `algorithme_vpp_cible.tex`.

1 Le problème en une page

On garde le planform du C-foil de l'Intens SY et l'on cherche la **section 2D** qui maximise la vitesse du bateau sur ses conditions d'usage, avec une section au moins aussi raide en flexion que l'actuelle.

La troisième colonne donne la notation de la note de M. Le Maître (version 01, 29/10/2026).

Symbole	Sens	Note Le Maître
$\boldsymbol{z} \in \mathbb{R}^{10}$	la section : ses 10 premiers scores PCA (§ 2)	u_f
$\boldsymbol{a} = (\text{cant}, \text{rake}, \text{yaw})$	l'attitude effective du foil, en degrés : *un état de fonctionnement*, pas de la géométrie	$z = (\theta_1, \theta_2, \theta_3)$
$\boldsymbol{x} = (\boldsymbol{z}, \boldsymbol{a}) \in \mathbb{R}^{13}$	l'entrée des modèles d'efforts	$x = (u_f, z)$
$\boldsymbol{h}(\boldsymbol{x}) = (C_1, \dots, C_7)$	le torseur $F_x, F_y, F_z, M_x, M_y, M_z$ et le $C_{p,\min}$ (cavitation), en coefficients, donnés par les codes	F
$\boldsymbol{c} = (\text{TWA}, \text{TWS})$, w_c	une condition de navigation et sa probabilité ($\sum_c w_c = 1$)	$v_w^{(k)}$, ω_k
$\boldsymbol{s}$	les 7 inconnues de la VPP : vitesse, dérive, gîte, assiette, déplacement (l'état), safran, « flat » (les réglages) ; l'attitude du foil en découle, $\boldsymbol{a} = A(\boldsymbol{s})$	$y = (u_r, a)$
$V^*(\boldsymbol{z}, \boldsymbol{c})$	la vitesse d'équilibre du bateau avec la section $\boldsymbol{z}$, à la condition $\boldsymbol{c}$	$V_k^\star(u_f; \mathsf{F})$
$J(\boldsymbol{z})$	l'objectif : la vitesse moyenne sur le profil d'usage	J_{moy}
$d_k = \partial V^* / \partial C_k$	la sensibilité de la vitesse optimisée à la sortie k (§ 4)	$b_N = B^\mathsf{T} \bar{\lambda}_N$

Théorie — le problème, écrit une fois

$$V^*(\boldsymbol{z}, \boldsymbol{c}) = \max_{\boldsymbol{s}} V(\boldsymbol{s}) \quad \text{tel que} \quad \boldsymbol{R}\big(\boldsymbol{s};\ \boldsymbol{h}(\boldsymbol{z}, A(\boldsymbol{s})),\ \boldsymbol{c}\big) = \boldsymbol{0}, \tag{1}$$

où $\boldsymbol{R}$ regroupe les 6 équations d'équilibre (forces et moments) du bateau entier : voiles, coque, safran et foil. Puis

$$\max_{\boldsymbol{z}} J(\boldsymbol{z}) = \sum_c w_c\, V^*(\boldsymbol{z}, \boldsymbol{c}) \qquad \text{tel que} \quad I_{xx}(\boldsymbol{z}) \geq I_{xx}(\text{section actuelle}). \tag{2}$$

Qu'est-ce qui coûte ? Les **codes** qui calculent $\boldsymbol{h}$: NPLLT × NeuralFoil (environ 0,5 s par calcul) et surtout AVL × XFOIL itéré (environ 17 s ; chiffres du 8e run). Or la VPP appelle le modèle du foil à *chaque itération* de son solveur, soit des centaines de fois par solve, pour chaque condition et chaque section essayée. Brancher les codes directement dans la VPP coûterait des centaines de calculs AVL par vitesse. D'où l'idée de la chaîne : **remplacer les codes par un modèle statistique de $\boldsymbol{h}$** (un solve prend alors quelques secondes), et ne lancer les codes que là où ils servent la décision.

Subtilité — pourquoi ce n'est pas une simple composition $f = g \circ h$

On peut écrire $J = g \circ \boldsymbol{h}$ comme Astudillo & Frazier [6] : $\boldsymbol{h}$ est cher, g (la VPP) est connue et rapide. Mais chez eux, $\boldsymbol{h}$ est évaluée au même point que l'objectif. Ici, la VPP a besoin de $\boldsymbol{h}$ en des attitudes $\boldsymbol{a} = A(\boldsymbol{s})$ qui sont *elles-mêmes des inconnues de l'équilibre*. Le modèle de $\boldsymbol{h}$ doit donc être bon sur une surface de l'espace $(\boldsymbol{z}, \boldsymbol{a})$ que l'on ne connaît qu'en résolvant la VPP. C'est la raison d'être du critère ciblé (§ 5, étape 3).

2 Brique 1 : la section

1. **16 poids de Kulfan** (8 par face), bord d'attaque arrondi, épaisseur au bord de fuite fixe (0,00197 de la corde). C'est la paramétrisation de NeuralFoil.
2. **Une famille admissible** : les poids dans une boîte centrée sur la section actuelle, et trois contraintes physiques. Les deux faces ne se croisent pas ; $I_{xx} \geq I_{xx}$ actuel ; NeuralFoil est confiant (au moins 0,90 à 0°, 2°, 4° et 6°).
3. **Une PCA sur les coordonnées** de 4 000 sections tirées dans la famille. On garde $k = 10$ modes, qui portent 99,87 % de la variance géométrique. La section est alors $\boldsymbol{z} \in \mathbb{R}^{10}$, et le décodage $\boldsymbol{z} \mapsto$ poids est exact.

Dans le code

`src/section_pca.py` : `KulfanBoxFamily` (la boîte et les contraintes), `SectionPCA` (`fit`, `encode`, `decode`). La taille de la boîte est l'option `--box-scale` de `scripts/run_real_vpp.py` : 0,5 aux runs 2 à 6, 0,75 au 7e, 3 au 8e. À l'échelle 3, la boîte ne limite plus : c'est I_{xx} qui retient la section.

3 Brique 2 : le modèle des efforts (7 co-krigeages)

Deux niveaux de fidélité, calculés aux mêmes points $\boldsymbol{x}$ (plan imbriqué) :

— niveau 1 (BF) : NPLLT × NeuralFoil, coût relatif 1 ;

— niveau 2 (HF) : AVL × XFOIL itéré, coût relatif 6. C'est la référence que la VPP utilise.

Un co-krigeage AR(1) par sortie (Kennedy & O'Hagan [1], sous la forme récursive de Le Gratiet [2]) :

$$Z_2(\boldsymbol{x}) = \rho\, Z_1(\boldsymbol{x}) + \delta(\boldsymbol{x}),$$

avec un noyau Matérn 5/2 et des hyperparamètres estimés par **MAP** : prior Inverse-Gamma(3, 2) sur les longueurs de corrélation, Log-Normale sur la variance et le bruit, $\mathcal{N}(1, 2^2)$ sur ρ. Il y a 7 co-krigeages indépendants, un par composante de $\boldsymbol{h}$, qui partagent le même plan.

À retenir — ce que les GP apprennent, et ce qu'ils n'apprennent pas

— Les GP apprennent $\boldsymbol{h}(\boldsymbol{z}, \boldsymbol{a})$ **directement**, à partir des sorties des codes, en des points $(\boldsymbol{z}, \boldsymbol{a})$ que nous choisissons. C'est une **régression ordinaire**, sortie par sortie.

— **Aucun GP n'est appris à travers l'équation d'équilibre.** On n'observe jamais V^* pour en déduire $\boldsymbol{h}$. L'équilibre ne sert qu'*après* l'apprentissage : pour prédire la vitesse (§ 4) et pour choisir les prochains calculs (§ 5).

Dans le code

`src/vpp_targeted.py` : `TorsorCoKriging` (les 7 `CoKriging` de mfego, ajustement, prédiction, conditionnement). Les codes : `src/foil_codes.py` (sur bdfoil_v1), lancés en parallèle par `CodeRunner` (`src/vpp_optimize.py`).

4 Brique 3 : la VPP et le profil d'usage

— **La VPP** est celle de MDS (Intens SY, foil « CFoilFSI »). Elle maximise la vitesse sous les 6 équations d'équilibre, avec 7 inconnues (vitesse, dérive, gîte, assiette, déplacement, safran, flat). Nous remplaçons seulement la surface de réponse du foil par **la moyenne** des 7 co-krigeages. Une correction de déformation du foil (FSI) s'y ajoute, mise à l'échelle par la raideur de la section.

— **Le profil d'usage** donne les conditions $\boldsymbol{c}$ et leurs poids w_c. Au 8e run, c'est `profile_v2.json` (dossier `profiles/`), avec 49 conditions, deux bosses à 40° et 80° de TWA, TWS de 5 à 17 nd, et des points sur les bords du domaine. Il se construit avec l'éditeur HTML du dossier `Bimodal-Sailing-Discretizer`.

Théorie — l'incertitude sur la vitesse, au premier ordre

À un état d'équilibre, une petite erreur δC_k change la vitesse de $d_k\,\delta C_k$, avec $d_k = \partial V^*/\partial C_k$ calculé par différences finies avec la VPP. Les 7 GP étant indépendants,

$$\operatorname{Var} V^* \;\approx\; \sum_k d_k^2\, s_k^2(\boldsymbol{x}),$$

avec s_k^2 la variance du co-krigeage k. Une erreur de traînée coûte beaucoup en vitesse ; une erreur d'effort latéral presque rien, car la VPP rattrape l'équilibre avec une dérive un peu différente.

Subtilité — deux limites à connaître

— **On met la moyenne dans la VPP**, alors que $\mathbb{E}[V^*(\boldsymbol{h})] \neq V^*(\mathbb{E}[\boldsymbol{h}])$ si la VPP est non linéaire. L'écart est du second ordre en s_k : il est négligeable là où le modèle est précis. Au 8e run, l'erreur de vitesse contre AVL × XFOIL est de 0,0006 nd à la meilleure section.

— **La VPP de MDS bute à 24 nd** : les surfaces de réponse de la coque ne vont pas plus loin. Au travers par vent fort, plusieurs conditions sont en butée et ne départagent pas les sections.

Dans le code — comment d_k est calculé, honnêtement

— **Une différence finie avant, à un seul pas.** On ajoute une constante à la sortie k du modèle, partout (environ 3 % de sa taille : 5e-4 pour F_x, 1e-2 pour F_y et F_z, 2e-2 pour M_x, 3e-3 pour M_y et M_z), on refait la VPP complète en partant de la base, et $d_k = (V_{\text{décalé}} - V_{\text{base}})/\text{pas}$. Toutes les inconnues sont réoptimisées : c'est la sensibilité de la vitesse *optimisée*.

— **12 solves par condition** : pour chaque k, la base est refaite dans le même processus que le solve décalé, car le solveur MDS dépend du chemin.

— **Seulement à la titulaire**, une mise à jour sur trois, puis la médiane des 3 derniers calculs. Un solve en échec garde l'ancienne valeur ; une condition en butée à 24 nd a $d = 0$.

— **Jamais vérifié** contre un autre pas, une différence centrée ou les multiplicateurs (§ 9).

Le code : la méthode `speed_sensitivities` de `IntensSYVPP` (`src/vpp_coupling.py`) et la méthode `sensitivities` de `VPPSectionOptimizer` (`src/vpp_optimize.py`). Le solve est `IntensSYVPP.solve` ; le profil, `OperatingProfile` (`src/operating_profile.py`).

5 Brique 4 : la boucle

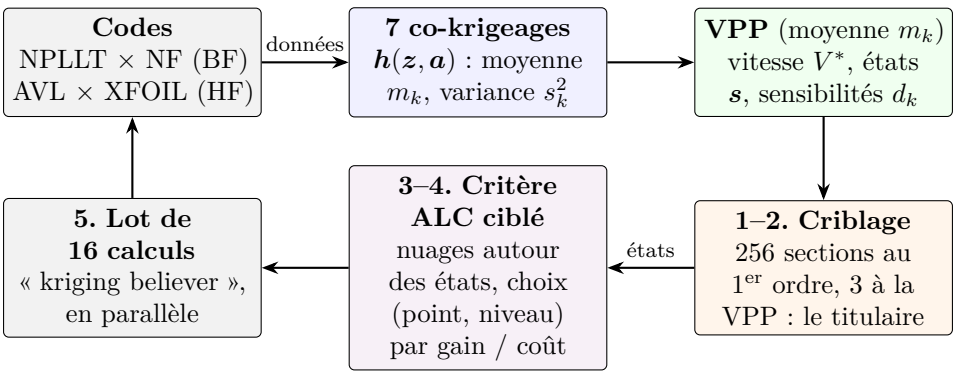


Figure 1 – Une mise à jour de la boucle. Les codes ne sont lancés qu'à l'étape 5 ; tout le reste utilise le modèle et la VPP.

Étape 0 — le plan de départ. 256 points BF : des sections de la famille, avec une attitude tirée par hypercube latin. Puis 64 points HF choisis par maximin parmi eux (plan imbriqué). On ajuste les 7 co-krigeages. La recherche part de la section actuelle (ou de la section admissible la plus proche).

Ensuite, à chaque mise à jour :

Étape 1 — cribler des sections. On tire 256 sections : la moitié dans une **région de confiance** autour du titulaire (la meilleure section du moment), l'autre dans toute la famille. Pour chacune, on

prédit au premier ordre le gain de vitesse μ et son incertitude σ, sans résoudre la VPP. On garde les 3 meilleurs scores $\mu + \sigma$: une section incertaine mais prometteuse a sa chance (comme UCB [9]).

Étape 2 — juger avec la VPP complète. Les 3 sections retenues et le titulaire passent à la VPP, sur toutes les conditions. La meilleure vitesse moyenne pondérée devient le titulaire. La région de confiance grandit (×1,5) si le titulaire a changé, et rétrécit (×0,7) sinon (TuRBO [8]).

Étape 3 — où le modèle doit être bon. Autour de chaque état d'équilibre de ces 4 sections, un nuage de 16 points : l'attitude bouge de ±1° en cant, ±0,3° en rake et ±1° en yaw, et la section aussi sur un point sur deux. Chaque point pèse $w_c\, d_k^2$: la probabilité de sa condition, fois l'effet de l'erreur sur la vitesse.

Étape 4 — choisir les calculs (le critère ALC ciblé). Pour chaque candidat $\boldsymbol{x}_m$ et chaque niveau l, on sait *avant de lancer le code* de combien la variance pondérée des nuages baissera : la variance d'un GP ne dépend pas des valeurs observées. On prend le meilleur rapport

$$(\boldsymbol{x}^\star, l^\star) = \arg\max_{m,\,l} \; \frac{\text{baisse de } \sum(\text{poids}) \times \text{variance sur les nuages}}{\text{coût du calcul}}.$$

C'est l'ALC de Cohn [4] et de Gramacy & Lee [5], avec la mesure d'intégration placée autour des équilibres au lieu de tout le domaine (Le Gratiet & Cannamela [3]).

Étape 5 — 16 calculs à la fois. On choisit 16 couples (point, niveau) l'un après l'autre. Après chaque choix, on fait comme si le calcul était fait (« kriging believer » [7]) : le choix suivant voit exactement la variance déjà retirée, et le lot s'étale de lui-même. Les 16 calculs tournent en parallèle, puis les vraies valeurs entrent dans le modèle.

Étape 6 — s'arrêter ou repartir. Si les 10 dernières mises à jour ont gagné moins de 0,01 nd en tout, la région a convergé : on garde son optimum local et l'on repart (au plus 2 fois) d'une section éloignée de tous les optima, en gardant toutes les données. La boucle s'arrête aussi à la limite de temps.

Étape 7 — vérifier. La meilleure section, la section de départ, les autres optima locaux et les meilleures sections des runs précédents sont recalculés avec les codes aux états VPP. On compare la vitesse du modèle à celle des codes, puis l'on analyse les sections sur le profil d'usage. Les coordonnées des sections (format Selig) vont dans `results/sections/`, les figures dans `figures/`.

Dans le code

`scripts/run_real_vpp.py` enchaîne tout (étapes 0 à 7) et écrit un dossier `runs/<date>_<nom>/`. La boucle est `VPPSectionOptimizer` (`src/vpp_optimize.py`) ; le critère, `choose` (`src/vpp_targeted.py`) et les gains exacts de `src/sequential_design.py` ; les figures, `scripts/real_run_figures.py`.

6 Ce qui ne sert qu'à aller vite

Ces couches ne changent pas la logique. Sans elles, l'algorithme est le même, en plus lent :

- les lots de 16 et les processus parallèles (28 pour les codes, 28 pour la VPP sur toute la machine) ;
- la mise à jour du facteur de Cholesky au lieu de tout refactoriser, pendant un lot ;
- les hyperparamètres réestimés seulement quand le plan a grossi de 25 %, et plus du tout au-delà de 2 500 points ;
- les sensibilités d_k recalculées une mise à jour sur trois (médiane des 3 dernières) ;
- la VPP avec la moyenne seule (sans variance) quand on n'a besoin que de la vitesse.

7 Les hypothèses essentielles

Hypothèse	Pourquoi	Ce qu'on risque
AR(1) à ρ constant, plan imbriqué	le modèle de [2], validé sur nos cas	une corrélation BF/HF qui varie dans le domaine
7 sorties indépendantes	un calcul donne tout le torseur ; simple	corrélations physiques ignorées (F_y et M_x viennent de la même portance)
hyperparamètres par MAP	stable quand il y a peu de données	résultat dépendant du prior au début du run
moyenne du modèle dans la VPP	la VPP reste déterministe	biais de second ordre là où le modèle est incertain
vitesse linéarisée (d_k)	rend le critère calculable	faux si l'erreur déplace beaucoup l'équilibre
intégration autour des états de 4 sections	seule zone utile à la décision	le reste du domaine peu appris
criblage au premier ordre	trier 256 sections sans 256 VPP	une bonne section mal classée
famille sans contrainte de forme (8e run)	ne pas brider l'optimum	une section à l'extrado non convexe

La liste complète (13 hypothèses) est dans `selection_points_vpp.tex`, § 7.

8 Où on en est

Run	Ce qui change	Gain sur la section actuelle
4e (23/10, 8 h)	recherche sur une distribution d'usage, recherche locale	+0,16 nd
5e (27/10, 2 h)	convergence, redémarrages, lots par Cholesky : même résultat en 2 h	+0,15 nd
6e (27/10, 6 h)	3 niveaux sur bdfoil_v1 : le niveau du milieu ne vaut pas son coût	+0,16 nd
7e (28/10, 8 h)	profil `profile_v1`, boîte 0,75 : un nouveau bassin	+0,21 nd
8e (29/10, 2 h 49)	profil `profile_v2`, boîte 3 (seule I_{xx} retient la section)	+0,26 nd

Chaque gain est mesuré sur le profil du run, contre AVL × XFOIL. Au 8e run, la section est moins cambrée (2,2 % à 55 % de corde, contre 4,2 % à 33 % pour l'actuelle). Elle gagne au travers et perd un peu au près serré par vent fort. Son extrado n'est plus convexe (creux de 3,6 ‰ de corde à 80 %) : c'est à vérifier.

9 Ce que nous devrions changer (note de M. Le Maître)

La différence principale : **notre ALC réduit la variance de la vitesse de chaque section**, prise une par une ; **son critère réduit le regret de conception**, la vitesse qu'on s'attend à perdre en choisissant mal. Le regret ne dépend que des écarts de moyenne entre sections candidates et de la variance de leurs *différences* $S_{ii} + S_{jj} - 2S_{ij}$: une incertitude commune ne vaut rien. Les changements, dans l'ordre :

1. **Les sensibilités par les multiplicateurs** du VPP, $d = B^{\mathsf{T}}\bar{\lambda}$ (sa 5.14) : un solve au lieu de 12 par condition, vérifié contre nos différences finies. Cela rend abordable une sensibilité b_i *par section candidate*, à son propre équilibre, au lieu de celle de la titulaire pour tout le monde.
2. **La loi jointe complète** des vitesses candidates, $S_{ij} = \sum_{c,c'} w_c w_{c'}\, b_{ic}^{\mathsf{T}} K_N(x^\star_{ic}, x^\star_{jc'})\, b_{jc'}$ (sa 12.6), entre sections *et* entre conditions, au lieu des seules variances au criblage.
3. **L'arrêt sur le regret**, $\tilde{r}_N = \mathbb{E}[\max_i \tilde{J}_i] - \max_i m_i \le \varepsilon_V$ (en nœuds), au lieu de « moins de 0,01 nd gagné en 10 mises à jour » : on s'arrête quand le choix est sûr, pas quand la recherche stagne. Les redémarrages restent : le regret ne voit que les candidates connues.
4. **Le critère de regret (sa 9.11) à la place de l'ALC**, en version par lot et à deux niveaux, comparé à l'ALC sur un cas test.
5. **Les hypothèses sur les entrées**, écrites : pas de vitesse V_f (coefficients, pas d'effet de Reynolds, sauf la correction FSI) ni d'enfoncement h_f (pas de surface libre dans AVL et XFOIL).

10 Questions ouvertes

1. **Les hyperparamètres par validation croisée (LOO)** au lieu du MAP, ou en diagnostic : les formules rapides existent pour le co-krigeage [3], et la LOO en forme fermée est déjà dans mfego (`CoKriging.leave_one_out`). La calibration de la variance compte, puisque le critère, le nôtre comme le sien, est fait de variances.
2. **Les sorties corrélées** : un modèle multi-sorties au lieu de 7 GP indépendants.
3. **L'incertitude à travers l'équilibre** : propager la loi de $\boldsymbol{h}$ dans la VPP au lieu de la moyenne.
4. **La forme** : faut-il une contrainte de convexité ou de régularité dans la famille ?
5. **La butée de 24 nd** de la VPP au travers par vent fort.

Références

[1] M. C. Kennedy, A. O'Hagan, *Predicting the output from a complex computer code when fast approximations are available*, Biometrika 87(1), 1–13, 2000.

[2] L. Le Gratiet, *Multi-fidelity Gaussian process regression for computer experiments*, thèse, Université Paris-Diderot, 2013.

[3] L. Le Gratiet, C. Cannamela, *Cokriging-based sequential design strategies using fast cross-validation techniques for multi-fidelity computer codes*, Technometrics 57(3), 418–427, 2015.

[4] D. Cohn, *Neural network exploration using optimal experiment design*, Neural Networks 9(6), 1996.

[5] R. B. Gramacy, H. K. H. Lee, *Adaptive design and analysis of supercomputer experiments*, Technometrics 51(2), 130–145, 2009.

[6] R. Astudillo, P. I. Frazier, *Bayesian optimization of composite functions*, ICML 2019.

[7] D. Ginsbourger, R. Le Riche, L. Carraro, *Kriging is well-suited to parallelize optimization*, in *Computational Intelligence in Expensive Optimization Problems*, Springer, 2010.

[8] D. Eriksson, M. Pearce, J. Gardner, R. Turner, M. Poloczek, *Scalable global optimization via local Bayesian optimization*, NeurIPS 2019.

[9] N. Srinivas, A. Krause, S. Kakade, M. Seeger, *Gaussian process optimization in the bandit setting : no regret and experimental design*, ICML 2010.